workflowval — Requirements

Nov 2, 2026 · @RK

workflowval is an open-source, model-agnostic CLI agent that reads an OpenAPI spec, infers multi-step workflows, writes and runs pytest tests against a live API, and triages every failure as a test bug, an API bug or spec drift.

Decisions before handoff

These are settled; coding agents read this section first.

Decision	Choice
Source of truth	The spec defines request and response shapes; the docs define business rules. When the spec and the docs contradict each other, the Reviewer files a CONFLICT bug report and does not pick a side (FR-14).
Input format	JSON only for every input: spec, auth config, workflow files, labels.
Data formats	`workflows/<id>.json`, `review.json`, `results.json`, `labels.json`, the auth config and the internal `.workflowval/state.json` are Pydantic models written in M0 and reviewed by you before M1 (format under FR-4). There is no plan file.
Agent roles	Planner, Reviewer and Writer, plus the code Runner; no role checks its own work (Architecture).
Loop limits	`--max-review-rounds` 2, `--max-fix-attempts` 3, `--max-iterations` 10, plus a step budget.
Human input	Optional; the pipeline runs end to end without it.
CI	Fails on any UNRESOLVED test, or on API_BUG / SPEC_DRIFT above a threshold. CONFLICT bug reports are listed in the report but do not fail CI.

Decision	Choice
Benchmark ownership	You write the seeded bugs and `labels.json`; agents never edit `benchmarks/seeded/`.
License	MIT.
Budget	No cost cap in v1: every run tracks and reports tokens and cost. A hard USD limit (--max-cost-usd) comes later. Loop and step limits still apply.
Spec validation	openapi-core validates live responses; openapi-spec-validator checks the spec file (FR-1). Both sit behind validate_response() in spec/, so a library can be swapped without touching the rest.

Overview

Existing tools mostly test one request at a time against the spec. workflowval targets the gap: **multi-step workflows and business rules** (create → update → cancel → verify refund), with an agent that explains every failure.

Goals

- Generate readable, runnable **pytest** tests for multi-step API workflows from an OpenAPI 3.x spec plus optional prose docs.
- Run them against a live base URL and iterate in an agent loop until each test passes or its failure is classified.
- Classify each failure as **TEST_BUG**, **API_BUG** or **SPEC_DRIFT**, with evidence and a suggested fix or bug report.
- Work with any LLM provider (Anthropic, OpenAI, Gemini, local models) behind one adapter.
- Run locally and in CI (GitHub Actions) with hard loop and step limits, and cost reported for every run.

Non-goals (v1)

- Security fuzzing or penetration testing (use RESTler or Schemathesis).
- Load or performance testing.
- GraphQL, gRPC, SOAP (REST over HTTP/JSON only).

- A web UI or hosted service.
- Generating tests from recorded traffic.
- YAML inputs (specs, workflows, config): JSON only in v1; convert YAML specs to JSON first.

Users and use cases

User	Use case	Example command
Backend developer	Get workflow tests for a new service before release	`workflowval test openapi.json --base-url http://localhost:8000`
QA / SDET	Find business-logic bugs and spec mismatches in an existing API	`workflowval test spec.json --docs docs/ --report out/`
Integration engineer	Check that a partner API behaves as its spec says	`workflowval drift partner-openapi.json --base-url https://sandbox.partner.com`
CI pipeline	Re-run generated tests on every PR; fail on new API bugs or unresolved tests	`workflowval run tests/wf --ci`

Functional requirements

ID	Requirement	Acceptance criteria
FR-1	**Load inputs:** OpenAPI 3.0/3.1 (JSON only for now, file or URL), optional docs folder (Markdown/HTML/text), base URL, auth config file (JSON). All inputs are JSON; YAML is not supported yet.	Loads Petstore and restful-booker specs; invalid spec gives a clear error naming the bad path; `$ref`s resolved.
FR-2	**Auth:** support API key (header/query), bearer token, basic auth, and a login workflow (call an endpoint, extract a token, reuse it, refresh on 401). Secrets come from env vars only.	restful-booker token flow works end to end; secrets never appear in logs, reports or generated tests.

ID	Requirement	Acceptance criteria
FR-3	**Spec model:** build an internal graph of resources, operations, parameters and producer → consumer links (e.g. `POST /bookings` returns `id` used by `GET /bookings/{id}`).	For Petstore, graph links `addPet.id` → `getPetById`; exported as JSON with `workflowval inspect`.
FR-4	**Workflow planning:** the agents propose workflows (CRUD lifecycle, auth, pagination, filtering, state transitions, negative paths) and business rules inferred from spec and docs. Output one reviewable `workflows/<id>.json` per workflow before writing tests.	Each workflow file lists its steps, expected status codes and assertions; `--plan-only` stops after this; user can edit the files and re-run.
FR-5	**Test generation:** write plain pytest + httpx files, one file per workflow, with fixtures for auth and test data, unique data per run, and cleanup steps. No custom runtime required to run them.	Generated tests run with plain `pytest` and no workflowval installed; `ruff` passes on output.
FR-6	**Execution:** run tests in a subprocess with timeouts, capture JSON results, request/response pairs and stdout.	Per-test result includes status, duration, failing assertion, last 3 HTTP exchanges.
FR-7	**Agent loop:** for each failing test, decide to fix the test, mark an API bug, or mark spec drift; re-run fixed tests. Each test gets at most N fix attempts (`--max-fix-attempts`, default 3); after N it is marked UNRESOLVED with its last failure and the loop moves on. The whole run stops after `--max-iterations` passes (default 10), or on any other budget.	The loop always terminates: every failure is classified, marked UNRESOLVED after N attempts, or the run reports which budget was hit. A test with `--max-fix-attempts 1` is patched at most once. Never edits tests to hide a real failure (assertion weakening is flagged).

ID	Requirement	Acceptance criteria
FR-8	**Failure triage:** every failure gets one label (TEST_BUG / API_BUG / SPEC_DRIFT, UNRESOLVED when fix attempts run out, or CONFLICT when the spec and docs disagree (FR-14)), a confidence (low/med/high), evidence (requests, responses, spec excerpt) and an action: patched test, bug report, or spec patch.	On the seeded-bug app, ⩾ 80% of seeded bugs labeled correctly (see Evaluation).
FR-9	**Bug reports:** for API_BUG, write a Markdown report with title, steps to reproduce (curl commands), expected vs actual, and spec reference.	Each report reproduces the bug when the curl steps are run by hand.
FR-10	**Spec drift:** for SPEC_DRIFT, propose a minimal JSON Patch (RFC 6902) to the spec. `workflowval drift` runs drift checks only.	Patch applies cleanly to the original spec and makes the observed response valid.
FR-11	**Reports:** `report.md` (human summary), `results.json` (machine), JUnit XML (CI).	GitHub Actions shows JUnit results; summary lists counts per label.
FR-12	**CLI:** `test`, `plan`, `run`, `drift`, `inspect`, `report`; flags for `--model`, `--role-model`, `--max-review-rounds`, `--max-fix-attempts`, `--max-iterations`, `--max-steps`, `--read-only`, `--allow-host`, `--ci`.	`workflowval --help` documents all; `--ci` exits non-zero on any UNRESOLVED test, or on API_BUG or SPEC_DRIFT above a threshold.
FR-13	**Resume and re-run:** state saved in `.workflowval/` (resume point and spec hash) so a run can resume; `run` re-executes existing tests without the model.	Killing a run mid-loop and re-running resumes at the last step.

ID	Requirement	Acceptance criteria
FR-14	**Spec–docs conflicts:** when the spec and the docs contradict each other (e.g. the spec allows page sizes up to 100, the docs say 50), the Reviewer writes a CONFLICT bug report in the FR-9 format: title, the spec excerpt and the docs excerpt, curl steps showing what the API actually does, expected per spec vs per docs, the affected workflows and a suggested resolution. Affected tests are marked xfail with the report link; no API_BUG or SPEC_DRIFT is raised on that rule until the report is resolved.	The planted conflict in the seeded app produces exactly one CONFLICT bug report and no API_BUG; report.md lists open CONFLICT reports.

Workflow files (FR-4)

Each workflow lives in its own file, `workflows/<id>.json`, and becomes exactly one test file, `tests/wf/test_<id>.py`. Steps name spec `operationId`s, `save` passes values to later steps, `rule` records the business rule checked and where it came from, `source` says who added it (planner, reviewer, user) and `basis` says whether the rule is documented or assumed.

```
{
  "id": "order_cancel_refund",
  "source": "reviewer",
  "basis": "documented",
  "locked": false,
  "rule": "Cancelling a paid order refunds the full amount (docs/orders.md)",
  "steps": [
    { "op": "createOrder", "save": { "order_id": "$.id", "paid_amount":
"$.total" }, "expect": { "status": 201 } },
    { "op": "payOrder", "params": { "id": "{order_id}" }, "expect": {
"status": 200 } },
    { "op": "cancelOrder", "params": { "id": "{order_id}" }, "expect": {
"status": 200 } },
    { "op": "getOrder", "params": { "id": "{order_id}" },
      "expect": { "status": 200, "body": { "$.state": "cancelled",
"$.refund.amount": "{paid_amount}" } } }
  ],
  "cleanup": [ { "op": "deleteOrder", "params": { "id": "{order_id}" } } ]
}
```

Workflows you write: add a file to `workflows/` with `"source": "user"` and `"locked": true`. Agents may never change or delete a locked file.

No plan file: the `workflows/` folder is the plan. Workflows are independent (each creates and cleans up its own data, NFR-3), so no run order is needed. The hash of the spec the workflows were built from lives in `.workflowval/state.json` (FR-13), and a spec change triggers re-planning. Rejected journeys are recorded in `review.json`, next to the proposal they answer.

Architecture

The agent is a pipeline of three roles (Planner, Reviewer, Writer) run one after another by loop/, so no role grades its own work. Each role is a prompt file plus a restricted tool set. Only the model adapter knows about providers; the loop, tools and domain logic are plain Python.

Module	Responsibility	Key interfaces
`models/`	Provider adapters behind one interface; default implementation via LiteLLM. A `FakeModel` replays scripted responses for tests.	`complete(messages, tools) -> Reply(text, tool_calls, usage)`
`tools/`	Functions the model can call, each with a Pydantic input schema: `read_spec`, `read_docs`, `read_workflows`, `write_workflow`, `write_review`, `answer_review`, `write_test`, `run_tests`, `read_results`, `http_probe` (guarded), `write_bug_report`, `propose_spec_patch`. `write_workflow` refuses locked files. Each role gets only the tools listed for it under Roles.	`Tool(name, description, schema, fn)`
`loop/`	The pipeline runner: runs roles in order, calls the model, validates tool args, executes tools, enforces budgets, review rounds and stop rules, checkpoints state.	`run(pipeline, budget) -> RunResult`; `Role(name, prompt, tools, model)`
`spec/`	OpenAPI loading, `$ref` resolution, dependency graph, response validation.	`SpecGraph`, `validate_response(op, resp)`
`planner/`	Prompts and post-processing that turn the graph and docs into `workflows/<id>.json` files.	`plan(graph, docs) -> list[Workflow]`
`gen/`	Prompts, templates and style rules for pytest output; formats with ruff.	`generate(workflow) -> TestFile`
`triage/`	Classification prompts plus deterministic checks (response vs schema, status code vs spec) that run before the model.	`classify(failure) -> Verdict`
`report/`	Markdown, JSON and JUnit writers.	`write_reports(run)`

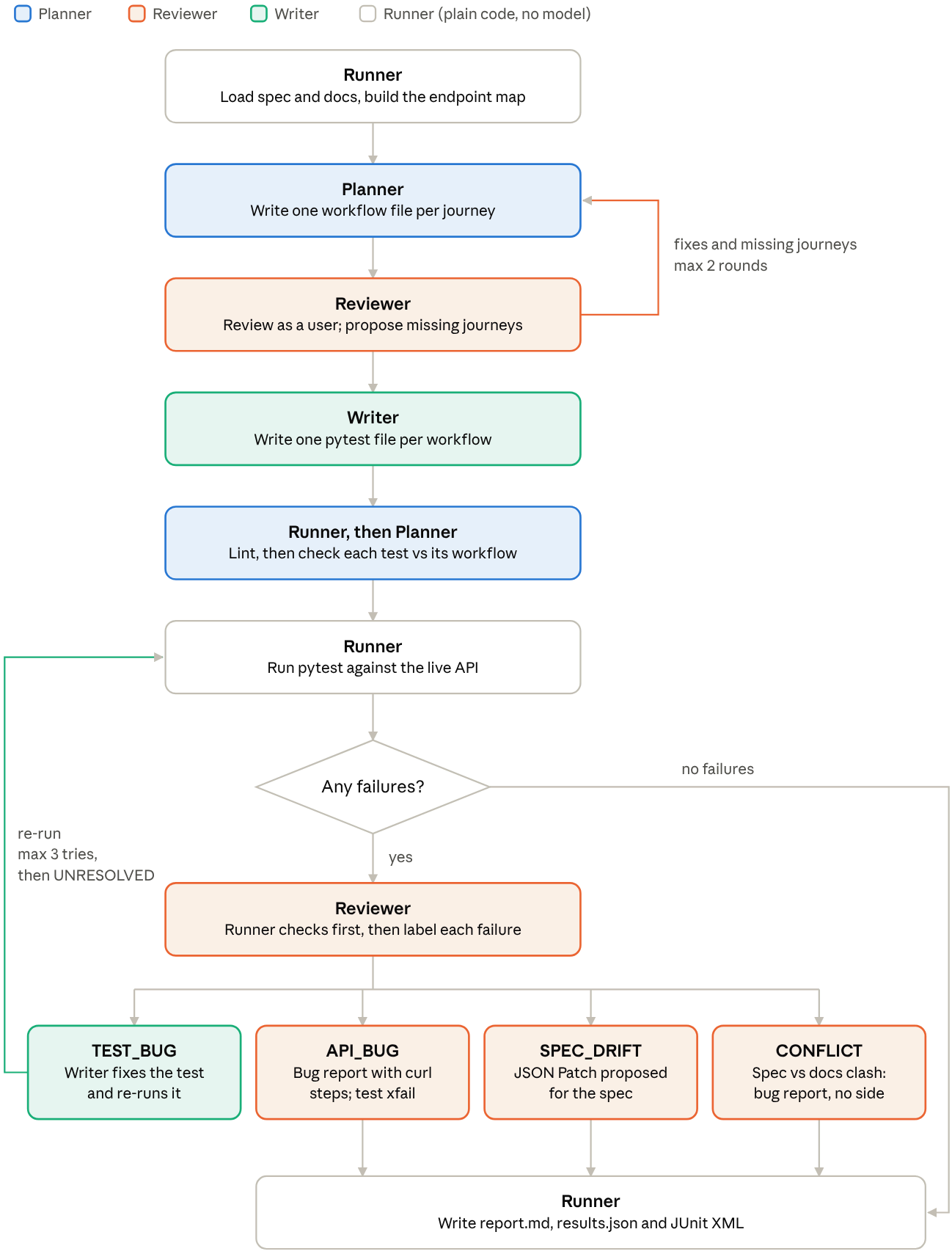
Module	Responsibility	Key interfaces
`cli.py`	Typer CLI wiring.	—

Roles

Role	Does	Can call	Never calls
Planner (A)	Writes and revises the workflow files; answers each proposed journey in `review.json`; reviews generated tests against their workflow files	`read_spec`, `read_docs`, `read_workflows`, `write_workflow`, `answer_review`	`write_test`, `http_probe`
Reviewer (B)	Reviews the workflow files as a user of the product: flags weak workflows and proposes missing user journeys; labels every failure; writes bug reports and spec patches	`read_spec`, `read_docs`, `read_workflows`, `write_review`, `read_results`, `http_probe`, `write_bug_report`, `propose_spec_patch`	`write_test`, `write_workflow`
Writer (C)	Writes one test file per workflow file; fixes only failures labeled TEST_BUG	`read_spec`, `read_workflows`, `read_results`, `write_test`	`write_workflow`, `write_bug_report`, `propose_spec_patch`
Runner (code, no model)	Lints with ruff, runs pytest, runs deterministic checks	—	—

Pipeline

Each role checks another role, and every loop back has a cap



workflowval run · 3 agents + Runner, 2 capped loops

Both loops back are capped: plan review at 2 rounds, test fixes at 3 tries, after which the test is UNRESOLVED. Every failure ends as exactly one of the four labels.

#	Step	Who	Limit
1	Load spec, build graph; compare the spec hash with `.workflowval/state.json` to decide whether to re-plan	Runner	—
2	Write one `workflows/<id>.json` per workflow (locked user files are kept as they are)	Planner	—
3	Review the workflow files as a user of the product: flag weak workflows and propose missing user journeys in `review.json`	Reviewer	—
4	Revise workflows: fix flagged ones; add each proposed journey as a file or reject it with a reason in `review.json`	Planner	`--max-review-rounds` (default 2)
5	Write one test file per workflow file	Writer	—
6	Lint with ruff, then review each test against its workflow file	Runner, then Planner	1 round
7	Run tests	Runner	per-test timeout
8	Deterministic checks (schema mismatch → likely SPEC_DRIFT; 5xx → likely API_BUG), then label each failure	Runner, then Reviewer	—
9	TEST_BUG: fix the test and re-run	Writer	`--max-fix-attempts` (default 3), then UNRESOLVED
10	API_BUG: bug report + mark test `xfail`; SPEC_DRIFT: spec patch	Reviewer	—

#	Step	Who	Limit
11	Write reports	Runner	—

Stop when every failure is classified or UNRESOLVED, or on `--max-iterations`, max steps, or no progress in 3 iterations.

Rules: the Writer fixes only failures the Reviewer labeled TEST_BUG; the Reviewer never edits tests; review rounds count toward the step budget.

Plan review from the user's side

The goal is no human in the loop: the Reviewer fills coverage gaps itself by thinking like the product's users, and the pipeline runs end to end without input. Human input (--plan-only edits, locked workflow files you write) is accepted but optional. The Reviewer reads the workflow files as someone who uses the product, not as a test linter. For each user named in the docs (or a generic API consumer if none), it asks: what would this user actually do, in what order, and what would they expect to happen? Its output, `review.json`, has two parts:

- **Fixes:** workflows that are wrong or weak (missing step, wrong expected status, assertion too loose), each with the reason.
- **Missing journeys:** user flows the workflows do not cover (e.g. a customer pays, cancels, then checks the refund; a retry after a timeout must not charge twice), each with the user, the steps and the rule it checks.

The Planner must add each proposed journey as a workflow file or reject it, recording its answer and reason next to the proposal in `review.json`; nothing is dropped silently. A journey whose rule comes from the docs or a locked user workflow is marked `"basis": "documented"`; one the Reviewer inferred is `"basis": "assumed"`. Assumed journeys are generated, run and triaged like any other; a failure on one is capped at medium confidence and listed in its own section of report.md, so readers can tell inferred rules from documented ones.

Model-agnostic rules: use only text, tool calls and JSON output. Treat provider extras (prompt caching, extended thinking) as optional flags. Model chosen by `--model provider/name` or `WORKFLOWVAL_MODEL`; each role can override it with --role-model (e.g. --role-model writer=provider/cheap-model) so the Writer can run on a cheaper model and the Reviewer on a stronger one. No code changes to swap.

Non-functional requirements

ID	Area	Requirement
NFR-1	Safety	Only call hosts in `--allow-host` (default: the base URL's host). Refuse non-localhost hosts unless `--allow-remote` is set.
NFR-2	Safety	`--read-only` blocks POST/PUT/PATCH/DELETE. Without it, print the planned write operations and require `--yes` (or `--ci`) before running against a non-local host.
NFR-3	Safety	Generated tests create their own data and clean it up; never delete or modify resources they did not create.
NFR-4	Cost	Hard limits on fix attempts per test, loop iterations and steps per run (`--max-steps`); track and print tokens and cost per run. A USD cap (`--max-cost-usd`) is deferred to a later release.
NFR-5	Portability	Python 3.11+; runs on macOS and Linux; no provider SDK imported outside `models/`.
NFR-6	Observability	Log every model call, tool call, arguments, result size, tokens and cost to `.workflowval/trace.jsonl`; `--verbose` echoes to console.
NFR-7	Secrets	Read secrets from env vars; redact tokens, API keys and `Authorization` headers from logs, reports and generated code.
NFR-8	Determinism	Seed and temperature configurable; `FakeModel` replay makes the loop fully deterministic in CI.
NFR-9	Robustness	Validate all tool arguments with Pydantic; on invalid output, return the validation error to the model (max 2 retries) instead of crashing.
NFR-10	Quality	Unit test coverage ⩾ 80% for `spec/`, `loop/`, `triage/`; `ruff` and `mypy --strict` clean.
NFR-11	License	MIT.

Evaluation

Test targets (run locally where possible, not against shared public instances)

Target	Why	Use for
Swagger Petstore (Docker)	Standard OpenAPI example	Smoke test of the whole pipeline
Restful-booker (local clone)	Practice API with known, documented bugs and a token auth flow	First real triage check; write a small OpenAPI spec for it if none exists
Seeded-bug app (`benchmarks/seeded/`)	Ground truth you control	Accuracy numbers for the README
RealWorld "Conduit" backends (2–3 implementations)	Same spec, many implementations, rich workflows	Headline demo: per-backend spec drift

Seeded-bug app: a small FastAPI service (orders: create, pay, cancel, refund) with an accurate spec, one planted spec–docs conflict (max page size 100 in the spec, 50 in the docs; expected label CONFLICT) and 10 deliberate defects, each tagged with its expected label:

- Wrong status code on validation error (API_BUG)
- Missing auth check on one write route (API_BUG)
- Off-by-one in pagination (API_BUG)
- Cancel allowed after shipping, breaking a documented rule (API_BUG)
- Refund amount wrong after partial cancel (API_BUG)
- Response field renamed but spec not updated (SPEC_DRIFT)
- Extra required field in spec that the API ignores (SPEC_DRIFT)
- Date returned in a different format than the spec says (SPEC_DRIFT)
- Idempotency key ignored on retry (API_BUG)
- 500 on an empty list filter (API_BUG)

Success metrics (v1 targets)

Metric	Target
Seeded bugs detected	⩾ 8 / 10
Triage label accuracy on detected bugs	⩾ 80%
False API_BUG reports on a clean build of the app	⩽ 1
Generated tests that run green on a clean build	⩾ 90%
Cost per full run on the seeded app	Reported per run (no target in v1)
Results across models	Publish a table for at least 2 providers
Plan coverage: seeded bugs with at least one workflow file that could catch them	10 / 10
Planted spec–docs conflict	1 CONFLICT bug report, 0 API_BUG on that rule

Milestones

Each milestone ends with a working demo and green CI. Give agents one milestone at a time.

M0 — Skeleton (½ day)

- [ ] Repo, `pyproject.toml` (uv), ruff, mypy, pytest, GitHub Actions CI
- [ ] Typer CLI with stub commands and `--help`
- [ ] `models/` interface, LiteLLM adapter, `FakeModel`
- [ ] `loop/` with budgets, stop rules and `trace.jsonl`; unit-tested with `FakeModel`
- [ ] `Role(name, prompt, tools, model)` and the pipeline runner in `loop/`, with per-role tool limits
- [ ] Pydantic models for every data format (workflow, review, results, labels, auth config, state); you review them before M1

M1 — Spec to passing tests (weekend 1)

- [ ] FR-1, FR-3: spec loading, `$ref` resolution, dependency graph, `workflowval inspect`
- [ ] FR-2: API key, bearer and login-flow auth

- [ ] FR-4: Planner writes CRUD workflows, one workflows/<id>.json each `; locked user files kept as they ar` e
- [ ] FR-5, FR-6: Writer generates pytest files; Runner executes them
- [ ] FR-7 (TEST_BUG only): Writer fix-and-rerun, capped by `--max-fix-attempts`
- [ ] Demo: `workflowval test` on Petstore and restful-booker produces a green suite

M2 — Reviewer triage (weekend 2)

- [ ] Seeded-bug app in `benchmarks/seeded/` with `labels.json`, written by you
- [ ] FR-8: Runner deterministic checks, then Reviewer labels each failure
- [ ] FR-9 bug reports, FR-10 spec patches, FR-11 reports, FR-14 conflict bug reports
- [ ] Benchmark script that scores labels and plan coverage against ground truth
- [ ] Demo: README table with seeded-bug results

M3 — User-side review, workflows and CI (weekend 3)

- [ ] Reviewer plan review from the user's side: `review.json`, missing journeys, `--max-review-rounds`
- [ ] Planner reviews each generated test against its workflow file
- [ ] Richer workflows: state transitions, pagination, negative paths, business rules from docs
- [ ] FR-12 `--ci` mode and `--role-model`; FR-13 resume
- [ ] GitHub Action (`uses: you/workflowval-action@v1`)
- [ ] NFR-1/2 safety guards fully enforced
- [ ] Demo: plan coverage on the seeded app with user-side review on vs off

M4 — Launch

- [ ] RealWorld multi-backend drift study (2–3 implementations) and write-up
- [ ] Cross-model results table (≥ 2 providers)
- [ ] README with quickstart, architecture, limits; publish to PyPI
- [ ] Budget limits: `--max-cost-usd` hard cap per run

Repo layout and conventions

```
workflowval/
  pyproject.toml
  src/workflowval/
    cli.py
    models/      # base.py, litellm_adapter.py, fake.py
    tools/
    loop/
    spec/
    planner/
    gen/         # templates/ for pytest output
    triage/
    report/
    prompts/     # versioned .md prompt files, never inline strings
  tests/         # unit tests (FakeModel only, no network)
  benchmarks/
    seeded/      # FastAPI app, openapi.json, labels.json
    targets.md   # how to run Petstore, restful-booker, RealWorld locally
  examples/
  .github/workflows/ci.yml
```

Stack: Python 3.11+, uv, Typer, Pydantic v2, httpx, pytest, LiteLLM, openapi-core and openapi-spec-validator for spec and response validation, ruff, mypy.

Rules for coding agents

- Work one milestone at a time; open a PR per checklist item with tests.
- No network calls in unit tests; use `FakeModel` and recorded HTTP fixtures.
- Prompts live in `prompts/` as files with a version header; changing a prompt requires re-running the benchmark.
- Never import a provider SDK outside `models/`.
- Every new tool needs a Pydantic schema, a docstring the model reads, and a unit test.
- Keep generated-test style rules in one place (`gen/style.md`) and enforce them with ruff.
- Never weaken or delete an existing assertion to make CI pass; flag it in the PR instead.
- Never edit `benchmarks/seeded/`: its bugs and `labels.json` are ground truth.
- A milestone is done only when its demo command passes in CI; you review and merge every PR.

Open questions

- [ ] Final name: workflowval (free on PyPI as of 2 Nov 2026; check GitHub before publishing).
- [ ] Reuse Schemathesis for single-request edge cases, or stay workflow-only in v1?
- [ ] Generate tests in one style only (pytest + httpx), or add a second target later (e.g. Postman collections)?